INTERNET APPLICATIONS AND ELECTRONIC PLATFORMS

RESEARCH REPORT

Generative AI as an Assistant for Developers

How ChatGPT and Similar Tools Help Beginners Understand and Build Web Applications

ASK	LEARN	TEST
Describe a problem	Understand the code	Verify the result

STUDENT NAME	UNIVERSITY ID
Menah Allah Mohamed ElSaeed Abdallah	2500104

Prepared for academic submission • 2026

A practical introduction to the opportunities and responsible use of AI in web development.

1. Introduction

Generative Artificial Intelligence (AI) is a type of technology that can create new content in response to instructions. Depending on the tool, this content may include written explanations, summaries, images, or computer code. Systems such as ChatGPT are increasingly used as learning assistants because they allow users to ask questions in everyday language and receive a response that can be refined through follow-up questions.

Web development is a useful area for exploring these tools. A basic website often combines HTML to structure information, CSS to control its appearance, and JavaScript to add interactive behavior. For a beginner, learning how these technologies work together can be challenging. An AI assistant may explain unfamiliar terms, generate small examples, and help the learner investigate errors. However, its output must be checked rather than accepted automatically.

The aim of this report is to describe how generative AI can support beginners who are learning web programming, present a simple example, and explain the benefits, risks, and good practices involved in using AI responsibly.

2. What Is Generative AI?

Generative AI refers to models trained to identify patterns in large collections of data and produce a new response based on a prompt. In a text-based assistant, a prompt is the question, instruction, or code supplied by the user. The model uses the prompt and the conversation context to generate a likely response. This can make it useful for explaining a concept in several ways or suggesting a starting point for a programming task.

Generative AI does not understand every situation perfectly, and it does not guarantee that generated information is true. It may produce plausible-sounding explanations that contain mistakes, omit important details, or rely on assumptions that do not match the user's project. For this reason, it is best viewed as a tool for support and exploration, while the learner remains responsible for checking the result.

3. How AI Helps in Web Development

3.1 Explaining code and concepts

A beginner can paste a short code fragment and ask for a line-by-line explanation. For example, an assistant can explain that an HTML element represents part of a page, a CSS selector identifies which elements should receive a style, and a JavaScript function groups instructions that can run when called. The learner can then ask for a simpler explanation or another example.

3.2 Generating practice examples

AI can provide a small starting example, such as a personal profile page, a navigation menu, a form, or a button that changes a message. Learners can request separate explanations of the HTML, CSS, and JavaScript, then change colors, labels, spacing, or behavior to observe what happens. This works best when examples are short enough to understand and test.

3.3 Finding and explaining errors

When a page does not behave as expected, the learner can share the relevant code and the error message. The assistant may identify common problems, such as a misspelled variable, a missing bracket, an incorrect element ID, or a script that runs before the page element is available. The suggested fix should be tested in the browser because several different issues can cause similar symptoms.

3.4 Planning and documentation

Before coding, an AI assistant can help list the pages a website needs, propose a simple development sequence, or turn a broad assignment into smaller tasks. It can also suggest meaningful variable names and draft comments that explain a function. These suggestions should be reviewed to ensure that the plan matches the assignment and the student's own design decisions.

4. A Practical Example

Imagine a student wants to make a web page with a heading and a button. When the user clicks the button, the heading should change. The following compact example uses HTML for the page elements and JavaScript for the interaction:

```
<h2 id="message">Welcome!</h2>
<button onclick="changeMessage()">Click me</button>

<script>
function changeMessage() {
  document.getElementById("message").textContent = "Hello, web developer!";
}
</script>
```

The heading has the ID message. When the button is clicked, the changeMessage() function runs. It finds the heading using document.getElementById() and replaces the heading text. A learner can ask an AI assistant to explain each line, then test the code in a browser and modify the message.

This example also illustrates an important learning strategy: ask for an explanation, run the code, observe the result, and experiment with a small change. Simply copying the code would provide less learning value than understanding how and why it works.

5. Benefits and Limitations

Benefits

- Immediate support: learners can ask questions while practising, without waiting for a scheduled lesson.
- Flexible explanations: a concept can be explained in simpler language, with an analogy, or through a different example.
- Faster troubleshooting: suggestions can help learners investigate likely causes of common errors.
- Project support: AI can help organize tasks, brainstorm ideas, and draft basic documentation.

Limitations and risks

- Incorrect answers: generated code may contain syntax errors, outdated approaches, or inaccurate explanations.
- Weak understanding: copying a complete solution without examining it can prevent the learner from developing independent skills.
- Security and privacy concerns: code or prompts may reveal sensitive details if users include passwords, API keys, personal data, or confidential information.
- Accessibility and quality: a generated page may not be usable by everyone or may ignore accessibility, performance, and maintainability requirements.

AI output should therefore be reviewed, tested, and compared with reliable technical documentation. For larger projects, security-sensitive features, or important decisions, additional review by a qualified person may be necessary.

6. Responsible Use for Students

Students can get more value from AI by treating it as a tutor rather than an answer machine. The following practices help protect learning quality and personal information:

- Ask specific questions, such as “Explain this function line by line” instead of requesting an unexplained complete project.
- Test examples in a safe environment and confirm that the result matches the intended behavior.
- Compare explanations with official documentation, such as MDN Web Docs, especially when learning browser APIs or JavaScript.
- Never include passwords, API keys, private student information, or confidential data in prompts.
- Follow the instructor's rules about AI use and disclose assistance when required.
- Make sure you can explain the final code and make at least some changes independently.

These practices keep the learner actively involved and help turn AI-generated suggestions into genuine understanding.

7. Conclusion

Generative AI tools such as ChatGPT can support beginners in web development by explaining code, producing practice examples, helping diagnose errors, and organizing project work. They can make learning more interactive because students can ask follow-up questions and request alternative explanations. Nevertheless, AI tools may make mistakes and cannot replace careful testing, trusted documentation, classroom guidance, or hands-on practice. The most effective approach is to use AI to support curiosity and problem-solving while verifying its output, protecting private information, and developing independent programming skills.

8. References

- OpenAI. “ChatGPT.” https://chatgpt.com/.
- MDN Web Docs. “Getting started with the web.” https://developer.mozilla.org/en-US/docs/Learn_web_development/Getting_started.
- MDN Web Docs. “JavaScript Guide.” https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide.
- W3C Web Accessibility Initiative. “Accessibility Fundamentals Overview.” https://www.w3.org/WAI/fundamentals/.
- GitHub Docs. “About GitHub and Git.” https://docs.github.com/en/get-started/start-your-journey/about-github-and-git.